MINISTÉRIO DA EDUCAÇÃO
UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ
CÂMPUS DOIS VIZINHOS
Curso de Engenharia de Software

ANDRÉ LUÍS ONYSZKO **SAMUEL RAMOS**
EDUARDO R. DOMINGUES **RAFAEL DARBERTO**

CCH BANCO DE DADOS

Transações e Segurança

DOIS VIZINHOS, 2025

1. Criando as tabelas:

Para criar as tabelas usamos o script fornecido pelo professor.

```sql
-- Cria as tabelas
CREATE TABLE clientes (
 id SERIAL PRIMARY KEY,
 nome TEXT NOT NULL
);


CREATE TABLE contas (
 id SERIAL PRIMARY KEY,
 cliente_id INTEGER REFERENCES clientes(id),
 saldo NUMERIC(12,2) NOT NULL
);

CREATE TABLE transferencias (
 id SERIAL PRIMARY KEY,
 conta_origem_id INTEGER REFERENCES contas(id),
 conta_destino_id INTEGER REFERENCES contas(id),
 valor NUMERIC(12,2) NOT NULL,
 data TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Query Query History

```sql
-- Cria as tabelas
CREATE TABLE clientes (
 id SERIAL PRIMARY KEY,
 nome TEXT NOT NULL
);


CREATE TABLE contas (
 id SERIAL PRIMARY KEY,
 cliente_id INTEGER REFERENCES clientes(id),
 saldo NUMERIC(12,2) NOT NULL
);

CREATE TABLE transferencias (
 id SERIAL PRIMARY KEY,
 conta_origem_id INTEGER REFERENCES contas(id),
 conta_destino_id INTEGER REFERENCES contas(id),
 valor NUMERIC(12,2) NOT NULL,
 data TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Data Output Messages Notifications

```
CREATE TABLE

Query returned successfully in 42 msec.
```

2. Populando dados:

Fizemos algumas inserções de clientes e suas respectivas contas, para que pudéssemos fazer as operações posteriores. Montamos também um select que retorna todas as contas com o nome de seus donos.

```sql
-- Popula as tabela
INSERT INTO clientes(nome) VALUES('Evandro');
INSERT INTO clientes(nome) VALUES('Andre');
INSERT INTO clientes(nome) VALUES('Samuel');
INSERT INTO clientes(nome) VALUES('Eduardo');

INSERT INTO contas(cliente_id, saldo) VALUES(1,1000);
INSERT INTO contas(cliente_id, saldo) VALUES(2,1000);
INSERT INTO contas(cliente_id, saldo) VALUES(3,1000);
INSERT INTO contas(cliente_id, saldo) VALUES(4,1000);

-- Consulta os dados inseridos
SELECT cli.nome, c.id, c.saldo FROM clientes cli JOIN contas c ON
c.cliente_id = cli.id;
```

Query Query History

```sql
-- Popula as tabela
INSERT INTO clientes(nome) VALUES('Evandro');
INSERT INTO clientes(nome) VALUES('Andre');
INSERT INTO clientes(nome) VALUES('Samuel');
INSERT INTO clientes(nome) VALUES('Eduardo');

INSERT INTO contas(cliente_id, saldo) VALUES(1,1000);
INSERT INTO contas(cliente_id, saldo) VALUES(2,1000);
INSERT INTO contas(cliente_id, saldo) VALUES(3,1000);
INSERT INTO contas(cliente_id, saldo) VALUES(4,1000);

-- Consulta os dados inseridos
SELECT cli.nome, c.id, c.saldo FROM clientes cli JOIN contas c ON c.cliente_id = cli.id;
```

Data Output Messages Notifications

Showing rows: 1 to 4 Page No: 1 of 1

	nome text	id integer	saldo numeric (12,2)
1	Evandro	1	1000.00
2	Andre	2	1000.00
3	Samuel	3	1000.00
4	Eduardo	4	1000.00

3. Criando a função de transferir:

Criamos uma função para automatizar transferências de saldo, passando o id da conta de origem, da conta de destino e o valor que será transferido. Tivemos que estudar na internet a questão da função retornar uma consulta, então depois de várias falhas, conseguimos. Montamos então uma consulta complexa para consultar a transferência feita trazendo o nome dos clientes das contas de destino e origem. Falaremos mais na conclusão sobre o uso do commit e rollback implicito na função.

```
-- Cria a funcao transferir com parametros de remetente destinatario e valor
CREATE OR REPLACE FUNCTION transferir(origem_id int, destino_id int, valor
int)
-- Retorna uma consulta (tabela com os dados da transferencia)
RETURNS TABLE(remetente TEXT, destinatario TEXT, quantidade NUMERIC, data
TIMESTAMP)
AS $$
DECLARE
    saldo_remetente int;
    id_da_transferencia int;
BEGIN
    -- Cadastra a transferencia:
    INSERT INTO transferencias(conta_origem_id, conta_destino_id, valor)
    VALUES(origem_id, destino_id, valor)
    -- Guarda o id gerado no cadastro na variavel:
    RETURNING transferencias.id INTO id_da_transferencia;

    -- Salva o saldo da conta de origem na variavel:
    SELECT saldo INTO saldo_remetente FROM contas WHERE id = origem_id;

    -- Se a conta de origem tem o valor, debita e credita
    IF saldo_remetente >= valor THEN
        UPDATE contas SET saldo = saldo - valor WHERE id = origem_id;
        UPDATE contas SET saldo = saldo + valor WHERE id = destino_id;

    -- Se nao tem o valor, lanca uma exceção
    ELSE
        RAISE EXCEPTION 'Saldo insuficiente na conta de origem';
        -- A PRÓPRIA FUNCTION FAZ UM ROLLBACK QUANDO LANÇADO EXCEPTION
    END IF;

    -- Retorna os dados da transferencia sucessida, A PROPRIA FUNCTION ao
finalizar sem exeção COMMITA
    RETURN QUERY
        SELECT clienteOrigem.nome as remetente, clienteDestino.nome as
destinatario, t.valor, t.data
        FROM transferencias t
        JOIN contas contaOrigem ON contaOrigem.id = t.conta_origem_id
        JOIN clientes clienteOrigem ON clienteOrigem.id =
contaOrigem.cliente_id
        JOIN contas contaDestino ON contaDestino.id = t.conta_destino_id
        JOIN clientes clienteDestino ON clienteDestino.id =
contaDestino.cliente_id
        WHERE t.id = id_da_transferencia;
END
```

```
$$ LANGUAGE plpgsql;
```

Query | Query History

```
    VALUES(origem_id, destino_id, valor)
    -- Guarda o id gerado no cadastro na variavel:
    RETURNING transferencias.id INTO id_da_transferencia;

    -- Salva o saldo da conta de origem na variavel:
    SELECT saldo INTO saldo_remetente FROM contas WHERE id = origem_id;

    -- Se a conta de origem tem o valor, debita e credita
    IF saldo_remetente >= valor THEN
        UPDATE contas SET saldo = saldo - valor WHERE id = origem_id;
        UPDATE contas SET saldo = saldo + valor WHERE id = destino_id;

    -- Se nao tem o valor, lanca uma exceção
    ELSE
        RAISE EXCEPTION 'Saldo insuficiente na conta de origem';
        -- A PRÓPRIA FUNCTION FAZ UM ROLLBACK QUANDO LANÇADO EXCEPTION
    END IF;

    -- Retorna os dados da transferencia sucessida, A PROPRIA FUNCTION ao finalizar sem exeção COMMITA
    RETURN QUERY
        SELECT clienteOrigem.nome as remetente, clienteDestino.nome as destinatario, t.valor, t.data
        FROM transferencias t
        JOIN contas contaOrigem ON contaOrigem.id = t.conta_origem_id
        JOIN clientes clienteOrigem ON clienteOrigem.id = contaOrigem.cliente_id
        JOIN contas contaDestino ON contaDestino.id = t.conta_destino_id
        JOIN clientes clienteDestino ON clienteDestino.id = contaDestino.cliente_id
        WHERE t.id = id_da_transferencia;
END
```

Data Output | Messages | Notifications

```
CREATE FUNCTION

Query returned successfully in 55 msec.
```

✓ Query returned successfully in 55 msec. ✕

Total rows: | Query complete 00:00:00.055 | CRLF | Ln 40, Col 21

4. Usando a função transferir:

Elaboremos dois usos da função, uma para ser bem sucecida e outra para falhar devido a falta de saldo na conta de origem.

```
select transferir(1, 2, 100);
-- Roda a função e da commit, pois a conta 1 possui 1000 de saldo
```

Query Query History

```
1  select transferir(1, 2, 100);
2  -- Roda a função e da commit, pois a conta 1 possui 1000 de saldo
```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

	transferir record
1	Evandro,Andre,100.00,2025-06-30 17:53:34.753753

```
select transferir(1, 2, 1001);
-- Roda a função mas da rollback, pois a conta 1 não 1001 de saldo
```

Query Query History

```
1  select transferir(1, 2, 1001);
2  -- Roda a função mas da rollback, pois a conta 1 não 1001 de saldo
```

Data Output Messages Notifications

```
ERROR:  Saldo insuficiente na conta de origem
CONTEXT:  função PL/pgSQL transferir(integer,integer,integer) linha 22 em RAISE

ERRO:  Saldo insuficiente na conta de origem
SQL state: P0001
```

5. Criando roles:

Fizemos a criação das roles como usuários, ou seja, que conseguem logar inserindo a senha. Atribuimos as respectivas permissões requisitadas no enunciado, além de obviamente permitir o usuário se conectar ao banco.

```
-- Cria um usuário auditor com senha
-- com permissão só de SELECT
CREATE ROLE auditor LOGIN PASSWORD 'auditor123';
GRANT SELECT ON clientes, contas, transferencias TO auditor;

-- Cria um usuário operator com senha
-- com permissão de INSERT e SELECT
CREATE ROLE operator LOGIN PASSWORD 'operator123';
GRANT INSERT, SELECT ON clientes, contas, transferencias TO operator;

-- Da permissão para ambos usuários conectarem ao banco
GRANT CONNECT ON DATABASE "cch-db2" TO operator, auditor;
```

Query Query History

```
1  -- Cria um usuário auditor com senha
2  -- com permissão só de SELECT
3  CREATE ROLE auditor LOGIN PASSWORD 'auditor123';
4  GRANT SELECT ON clientes, contas, transferencias TO auditor;
5
6  -- Cria um usuário operator com senha
7  -- com permissão de INSERT e SELECT
8  CREATE ROLE operator LOGIN PASSWORD 'operator123';
9  GRANT INSERT, SELECT ON clientes, contas, transferencias TO operator;
10
11 -- Da permissão para ambos usuários conectarem ao banco
12 GRANT CONNECT ON DATABASE "cch-db2" TO operator, auditor;
```

Data Output Messages Notifications

```
GRANT

Query returned successfully in 44 msec.
```

6. Usando role auditor:

Utilizamos o SET ROLE para simular a atuação do usuário no banco. Seguimos uma linha de raciocínio para testar ambas os dois usuários: criar um novo cliente, uma nova conta para esse cliente e então cadastrar uma transferência com essa nova conta, para depois consultar.

```
SET ROLE auditor; -- Troca para role auditor
```

Query Query History

```
1 SET ROLE auditor; -- Troca para role auditor
```

Data Output Messages Notifications

```
SET

Query returned successfully in 32 msec.
```

```
-- Não Funcional: tenta fazer um cadastro de cliente, contas e transferencias
-- (a role auditor tem permissão para INSERT)
INSERT INTO clientes(nome) VALUES('Rafael');
INSERT INTO contas(cliente_id, saldo) VALUES(5, 1000);
INSERT INTO transferencias(conta_origem_id, conta_destino_id, valor)
VALUES(5, 1, 100);
```

Query Query History

```
1 -- Não Funcional: tenta fazer um cadastro de cliente, contas e transferencias
2 -- (a role auditor tem permissão para INSERT)
3 INSERT INTO clientes(nome) VALUES('Rafael');
4 INSERT INTO contas(cliente_id, saldo) VALUES(5, 1000);
5 INSERT INTO transferencias(conta_origem_id, conta_destino_id, valor) VALUES(5, 1, 100);
```

Data Output Messages Notifications

```
ERROR:  permissão negada para tabela clientes

ERRO:  permissão negada para tabela clientes
SQL state: 42501
```

```
-- Funcional: faz uma consulta utilizando as três tabelas
SELECT clienteOrigem.nome as remetente, clienteDestino.nome as destinatario,
t.valor, t.data
FROM transferencias t
JOIN contas contaOrigem ON contaOrigem.id = t.conta_origem_id
JOIN clientes clienteOrigem ON clienteOrigem.id = contaOrigem.cliente_id
JOIN contas contaDestino ON contaDestino.id = t.conta_destino_id
JOIN clientes clienteDestino ON clienteDestino.id = contaDestino.cliente_id;
```

Query Query History

```
1 -- Funcional: faz uma consulta utilizando as três tabelas
2 SELECT clienteOrigem.nome as remetente, clienteDestino.nome as destinatario, t.valor, t.data
3 FROM transferencias t
4 JOIN contas contaOrigem ON contaOrigem.id = t.conta_origem_id
5 JOIN clientes clienteOrigem ON clienteOrigem.id = contaOrigem.cliente_id
6 JOIN contas contaDestino ON contaDestino.id = t.conta_destino_id
7 JOIN clientes clienteDestino ON clienteDestino.id = contaDestino.cliente_id;
```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

	remetente text	destinatario text	valor numeric (12,2)	data timestamp without time zone
1	Evandro	Andre	100.00	2025-06-30 17:53:34.753753

7. Usando a role operator:

Como no tópico anterior, usemos o SET ROLE para simular a atuação do usuário no banco e seguimos a mesma linha de raciocínio. Entre tanto no meio do teste se deparamos com um quisito que esquecemos de se atentar: como todas as tabelas que estamos fazendo registros usam uma chave primaria serializada, temos que delegar o uso da tabela de serialização ao usuário operator, não só adianta delegar o insert. Então concertemos e prosseguimos.

```
SET ROLE operator; -- Troca para role auditor
```

Query Query History

```
1  SET ROLE operator; -- Troca para role auditor
```

Data Output Messages Notifications

```
SET

Query returned successfully in 45 msec.
```

```
-- Não Funcional: tenta fazer um cadastro de cliente, contas e transferencias
-- (a role operator tem permissão para INSERT mas não às tabelas de
sequencias)
INSERT INTO clientes(nome) VALUES('Rafael');
INSERT INTO contas(cliente_id, saldo) VALUES(5, 1000);
INSERT INTO transferencias(conta_origem_id, conta_destino_id, valor)
VALUES(5, 1, 100);
```

Query Query History

```
1  -- Não Funcional: tenta fazer um cadastro de cliente, contas e transferencias
2  -- (a role operator tem permissão para INSERT mas não às tabelas de sequencias)
3  INSERT INTO clientes(nome) VALUES('Rafael');
4  INSERT INTO contas(cliente_id, saldo) VALUES(5, 1000);
5  INSERT INTO transferencias(conta_origem_id, conta_destino_id, valor) VALUES(5, 1, 100);
```

Data Output Messages Notifications

```
ERROR:  permissão negada para sequência clientes_id_seq

ERRO:  permissão negada para sequência clientes_id_seq
SQL state: 42501
```

```
RESET ROLE; -- volta a role postgres
GRANT USAGE, SELECT, UPDATE ON SEQUENCE clientes_id_seq, contas_id_seq,
transferencias_id_seq TO operator;
SET ROLE operator; -- Troca para role auditor
```

Query Query History

```
1  RESET ROLE; -- volta a role postgres
2  GRANT USAGE, SELECT, UPDATE ON SEQUENCE clientes_id_seq, contas_id_seq, transferencias_id_seq TO operator;
3  SET ROLE operator; -- Troca para role auditor
```

Data Output Messages Notifications

```
SET

Query returned successfully in 34 msec.
```

```sql
-- Funcional: tenta fazer um cadastro de cliente, contas e transferencias
-- (a role operator tem permissão para INSERT e às tabelas de sequencias)
-- Obs.: tem que rodar um por um
INSERT INTO clientes(nome) VALUES('Rafael');
INSERT INTO contas(cliente_id, saldo) VALUES(5, 1000);
INSERT INTO transferencias(conta_origem_id, conta_destino_id, valor)
VALUES(5, 1, 100);
```

Query Query History

```sql
-- Funcional: tenta fazer um cadastro de cliente, contas e transferencias
-- (a role operator tem permissão para INSERT e às tabelas de sequencias)
-- Obs.: tem que rodar um por um
INSERT INTO clientes(nome) VALUES('Rafael');
INSERT INTO contas(cliente_id, saldo) VALUES(5, 1000);
INSERT INTO transferencias(conta_origem_id, conta_destino_id, valor) VALUES(5, 1, 100);
```

Data Output Messages Notifications

```
INSERT 0 1

Query returned successfully in 33 msec.
```

```sql
-- Funcional: faz uma consulta utilizando as três tabelas
SELECT clienteOrigem.nome as remetente, clienteDestino.nome as destinatario,
t.valor, t.data
FROM transferencias t
JOIN contas contaOrigem ON contaOrigem.id = t.conta_origem_id
JOIN clientes clienteOrigem ON clienteOrigem.id = contaOrigem.cliente_id
JOIN contas contaDestino ON contaDestino.id = t.conta_destino_id
JOIN clientes clienteDestino ON clienteDestino.id = contaDestino.cliente_id;
```

Query Query History

```sql
-- Funcional: faz uma consulta utilizando as três tabelas
SELECT clienteOrigem.nome as remetente, clienteDestino.nome as destinatario, t.valor, t.data
FROM transferencias t
JOIN contas contaOrigem ON contaOrigem.id = t.conta_origem_id
JOIN clientes clienteOrigem ON clienteOrigem.id = contaOrigem.cliente_id
JOIN contas contaDestino ON contaDestino.id = t.conta_destino_id
JOIN clientes clienteDestino ON clienteDestino.id = contaDestino.cliente_id;
```

Data Output Messages Notifications

Showing rows: 1 to 2 Page No: 1 of 1

	remetente text	destinatario text	valor numeric (12,2)	data timestamp without time zone
1	Evandro	Andre	100.00	2025-06-30 17:53:34.753753
2	Rafael	Evandro	100.00	2025-06-30 17:59:46.992128

8. Revogando permissões e deletando as roles:

Para deletar um usuário temos que revogar todas suas permissões antes, então basicamente retrocedemos o processo de delegação das permissões, retirando todas as permissões dos usuários sobre as tabelas de sequência, das tabelas clientes, contas e transferência, e por último o acesso a conexão com o banco de dados. Somente após isso, se torna possível deletar a role.

```
-- Voltando a role postgres
RESET ROLE;

-- Remove todas as permissões do auditor e do operator sobre as tabelas de
sequencia
REVOKE ALL ON
clientes_id_seq, contas_id_seq, transferencias_id_seq
FROM auditor, operator;

-- Remove todas as permissões do auditor e do operator sobre as demais
tabelas
REVOKE ALL ON
clientes, contas, transferencias
FROM auditor, operator;

-- Remove a permissão de conexão do auditor e operator
REVOKE CONNECT ON DATABASE "cch-db2" FROM auditor, operator;
```

Query Query History

```
1  -- Voltando a role postgres
2  RESET ROLE;
3
4
5  -- Remove todas as permissões do auditor e do operator sobre as tabelas de sequencia
6  REVOKE ALL ON
7  clientes_id_seq, contas_id_seq, transferencias_id_seq
8  FROM auditor, operator;
9
10
11 -- Remove todas as permissões do auditor e do operator sobre as demais tabelas
12 REVOKE ALL ON
13 clientes, contas, transferencias
14 FROM auditor, operator;
15
16
17 -- Remove a permissão de conexão do auditor e operator
18 REVOKE CONNECT ON DATABASE "cch-db2" FROM auditor, operator;
```

Data Output Messages Notifications

```
REVOKE

Query returned successfully in 37 msec.
```

```
-- Deleta as roles
DROP ROLE auditor, operator;
```

Query Query History

```
1  -- Deleta as roles
2  DROP ROLE auditor, operator;
```

Data Output Messages Notifications

```
DROP ROLE

Query returned successfully in 35 msec.
```

Considerações Finais

Durante o trabalho aplicamos tudo que foi visto durante as aulas de transações e segurança, e também acabamos revisando o conteúdo de funções em plpgsql.

1. Optamos pela aplicação da função para as transações pois é o que mais faz sentido em um cenário real de desenvolvimento, e além disso nos permitiu utilizar o fluxo condicional, os IFs, inicialmente pensando em usufruir delas para dado uma condição, executasse o COMMIT, e se falhasse rodaria o ROLLBACK. O comando de cadastro da transferência foi posicionado estrategicamente dentro da função, antes do possível rollback, para ter certeza de que as instruções tavam sendo revertidas.

2. Para nossa surpresa, se deparamos com a seguinte mensagem quando executamos a função pela primeira vez:

   ERROR: encerramento de transação inválido
   CONTEXT: função PL/pgSQL transferencia(integer,integer,integer) linha 12 em COMMIT ERRO: encerramento de transação inválido
   SQL state: 2D000

   E buscando na internet descobrimos que o Postgresql não aceita os comandos COMMIT e ROLLBACK dentro de uma function, pois ele mesmo executa implicitamente esses comandos. Quando a função finaliza sem nenhum lançamento de exceção, ele roda o COMMIT, e caso haja qualquer exceção, ele roda o ROLLBACK e reverte tudo aquilo já feito dentro da função.

3. Optamos por continuar com a função, executando de forma implicita os comandos de transações, já que o enunciado nos permitiu isso, e acreditemos que talvez até seria o intuíto do professor que a gente descobrisse esse comportamento e prática.

4. Sobre o conteúdo de segurança, não tivemos nenhuma novidade, desenvolvemos de forma bem facil. Somente tivemos o caso isolado de não ter se atentado a delegação do seq_id, mas não foi nenhum impedimento, bastou delegar posteriormente as devidas permissões aos usuários.